CHAPTER 6 — SOFTWARE METRICS

Descriptive & Definitional Questions

Section 1: Measurement Fundamentals, Frameworks & Categories

Q6.D01. Define software metric, and explain why measurement is essential in software engineering.

A software metric is a quantitative measure of the degree to which a software system, component, or development process possesses a given property or attribute. In standard software engineering terminology:

- **Measure:** Provides a quantitative indication of the extent, amount, dimension, capacity, or size of some attribute of a product or process.
- **Measurement:** The formal act of determining a measure.
- **Metric:** A quantitative measure of the degree to which a system, component, or process possesses a given attribute (often normalizing or relating measures).
- **Indicator:** A metric or combination of metrics that provides insight into the software process, project, or product, enabling effective decision-making.

Why measurement is essential in software engineering:

1. **Quantitative Visibility and Control:** Without measurement, software development is subjective and intuitive; as Tom DeMarco stated: "*You cannot control what you cannot measure.*" Metrics provide objective baseline visibility into schedule, budget, and quality.
2. **Project Planning and Cost Estimation:** Historical metric baselines (e.g., productivity rates, defect rates, size measures) enable accurate estimation of cost, effort, and staffing for future projects.
3. **Quality Evaluation and Defect Containment:** Metrics quantify code complexity, coupling, and defect density, allowing teams to identify defect-prone modules early before deployment.
4. **Process Improvement:** Metrics reveal operational bottlenecks across lifecycle phases, enabling organizations to evaluate whether adopting a new methodology, tool, or training program genuinely improves productivity and product reliability.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 250–252; Companion Slides 2–4. Exercise 6.1.

Q6.D02. Discuss the primary areas of application for software metrics, and identify common problems encountered during their organizational implementation.

Primary Areas of Application for Software Metrics:

1. **Cost and Effort Estimation:** Using size and complexity metrics (e.g., LOC, Function Points, Use Case Points) to parameterize models like COCOMO to forecast delivery timelines and financial budgets.
2. **Quality Assurance and Defect Modeling:** Tracking defect injection rates, defect removal efficiency (DRE), and cyclomatic complexity to allocate testing effort to the most error-prone modules.
3. **Productivity and Benchmarking:** Evaluating engineering efficiency (LOC/PM, FP/PM) across projects and comparing performance against commercial industry standards.
4. **System Architecture and Maintenance:** Assessing structural health using coupling, cohesion, span, and inheritance metrics to identify candidate modules for refactoring before system decay occurs.

Common Problems Encountered During Implementation:

1. **Lack of Clear Measurement Goals:** Collecting massive volumes of metric data without a well-defined purpose or target question (violating the GQM paradigm), resulting in "data cemeteries."
2. **Human Resistance and Fear:** Developers frequently perceive metrics programs as punitive surveillance tools designed to evaluate individual employment performance rather than process capability.
3. **Incentive-related measurement risk:** A narrow reward target may encourage people to optimize the reported number instead of the intended outcome. The LOC and ticket examples are possible scenario risks, not guaranteed effects or claims attributed to a named law.
4. **High Overhead and Tooling Friction:** Manual metric collection imposes administrative drag on developers; without automated telemetry integrated into the CI/CD pipeline, data collection quickly lapses.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 252–254; Companion Slides 7–8. Exercise 6.2.

Q6.D03. Describe the three major categories of software metrics with representative examples of each.

Software metrics are categorized into three distinct operational domains:

1. **Product Metrics:**
2. **Definition: Measures that quantify the characteristics of any deliverable or intermediate artifact generated during the software lifecycle**, including source code, design models, requirements specifications, and test cases.
3. **Examples:** Lines of Code (LOC), Function Points (FP), McCabe's Cyclomatic Complexity $V(G)$, Halstead's Volume V, Chidamber & Kemerer's Coupling Between Objects (CBO), and Mean Time To Failure (MTTF).
4. **Process Metrics:**
5. **Definition: Measures that quantify the efficacy, efficiency, and characteristics of the engineering activities, development lifecycle methods, and tools used to build the software.**
6. **Examples:** Defect Removal Efficiency (DRE), defect discovery rate across phases, phase containment effectiveness, average bug turnaround time, and process maturity level (CMMI).
7. **Project Metrics:**
8. **Definition: Measures that describe the project operational status, resource expenditure, and execution progress**, used by managers to guide day-to-day development and track milestones.
9. **Examples:** Total project effort expended (Person-Months), actual versus budgeted cost, schedule slip / milestone variance, team staffing allocation, and overall project productivity (FP/PM).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 254–255; Companion Slides 9–10. Exercise 6.3.

Q6.D04. Explain the Goal–Question–Metric (GQM) paradigm and illustrate its application with a concrete software engineering example.

The Goal–Question–Metric (GQM) approach is a goal-oriented, top-down measurement framework that links a defined goal to questions and then to metrics.

The Three Hierarchical Levels:

1. **Conceptual Level (Goal):** A goal is defined for an object (product, process, or resource) with respect to a particular quality focus, from a specific viewpoint, within a defined environmental context.
2. **Operational Level (Question):** A set of specific questions is formulated to characterize the assessment, diagnosis, or achievement of the stated goal.
3. **Quantitative Level (Metric):** A set of objective or subjective data metrics is associated with each question to provide quantitative answers.

Concrete Example:

- **Goal:** *Improve the quality of the payment processing module from the maintenance team's perspective.*
- **Question 1:** *What is the current defect density of the payment module?*
- **Metric 1.1:** Number of post-release defects per KLOC.
- **Metric 1.2:** Defect Removal Efficiency (DRE) of the unit test suite.
- **Question 2:** *Is the module structurally too complex to maintain?*
- **Metric 2.1:** Average Cyclomatic Complexity ($V(G)$) of payment methods.
- **Metric 2.2:** Coupling Between Objects (CBO) of the payment class.

Source: Leach, R., "Introduction to Software Engineering", printed pp. 39–40.

Q6.D05. Define the five measurement scales used in software measurement theory and explain the representation condition and meaningfulness.

Measurement assigns numerical values to empirical attributes under stated rules. Stevens' classic taxonomy contains four scales—nominal, ordinal, interval, and ratio. Software-measurement treatments also use the absolute scale as a fifth type. The scale constrains which numerical statements are meaningful:

1. **Nominal:** Unordered categories; equality/inequality and counts are meaningful. Example: defect type.
2. **Ordinal:** Ordered ranks, but intervals are not known to be equal; medians/ranks are meaningful, arithmetic differences are not. Example: defect severity.
3. **Interval:** Equal intervals with an arbitrary zero; differences are meaningful, ratios are not. Example: calendar dates.
4. **Ratio:** Equal intervals and a true zero; ratios are meaningful. Examples: LOC and execution time, when counted consistently.
5. **Absolute:** Direct counts with one possible mapping. Example: number of function parameters.

Representation Condition and Meaningfulness:

- **Representation condition:** The numerical mapping preserves the empirical relations it represents, such as ordering.
- **Meaningfulness:** A statement is meaningful when its truth remains invariant under transformations allowed by the scale. An arithmetic mean of ordinal ranks is not justified; ratio comparisons require a ratio scale.

Source: Stevens, "On the Theory of Scales of Measurement," pp. 677–680; Fenton, "Software Measurement: A Necessary Scientific Basis," pp. 200–201.

Q6.D06. Explain the distinction between Predictor Metrics and Control Metrics, and between Internal and External Software Attributes.

Distinction Between Internal and External Attributes:

- **Internal Attributes:**
- **Definition: Properties of a software entity that can be measured purely in terms of the entity itself, independent of its runtime behavior or external environment.**
- **Examples:** Source code size (LOC), structural cyclomatic complexity ($V(G)$), modular coupling (CBO), and cohesion (LCOM).
- **External Attributes:**
- **Definition: Properties of a software entity that can only be evaluated with respect to how the entity interacts with its operational environment, users, or hardware.**
- **Examples:** Reliability (MTTF), maintainability, usability, portability, and user-perceived performance.

Distinction Between Predictor Metrics and Control Metrics:

- **Predictor Metrics:**
- **Definition: Metrics that measure current internal attributes to forecast future internal or external attributes.**
- **Mechanism:** Internal measures are used as independent variables in empirical models to estimate external quality.
- **Example:** Measuring cyclomatic complexity and code churn during development to predict post-release defect density and warranty repair effort.
- **Control Metrics:**
- **Definition: Metrics that provide real-time managerial visibility into active engineering processes to ensure projects execute within planned constraints.**
- **Mechanism:** Tracks process variables against preset organizational baselines and thresholds.
- **Example:** Monitoring defect discovery rate per inspection hour, requirements volatility, and milestone burn-down rates.

Source: Sommerville, I., "Software Engineering", 9th ed., printed pp. 668–671.

Section 2: Size Metrics & Halstead's Software Science

Q6.D07. What are the counting rules, strengths, and primary technical criticisms of Lines of Code (LOC) as a size and productivity metric?

Lines of Code (LOC / KLOC) Counting Rules:

- **Physical LOC:** Counts every physical line in the source file, including comments and blank lines.
- **Logical LOC:** Counts executable statements, declarative statements, and data definitions, explicitly excluding blank lines and standalone comments.

Strengths of LOC:

1. **Intuitive and Tangible:** Directly understandable by engineers and managers with no mathematical training.
2. **Automated and Objective:** Can be counted rapidly and automatically using static analysis tools and compiler preprocessors.
3. **Extensive Historical Data:** Decades of published benchmark data and empirical cost estimation models (such as COCOMO) rely on LOC as their primary sizing parameter.

Primary Technical Criticisms:

1. **Language Dependence and Inverse Productivity Paradox:** High-level, expressive languages (e.g., Python, SQL) achieve the same computational task in far fewer lines of code than low-level languages (e.g., C, Assembly). Computing productivity as LOC/PM creates a false paradox where verbose, inefficient programming appears more productive.
2. **Lack of Early Availability:** LOC cannot be directly counted during requirements analysis or preliminary architecture; it can only be estimated, introducing significant uncertainty into early project bids.
3. **Indifference to Design Complexity:** LOC treats a simple sequential assignment statement identically to a deeply nested, mathematically complex recursive algorithm.
4. **Discourages Code Reuse and Refactoring:** Developers evaluated on LOC are penalized for reusing existing libraries or refactoring complex classes into concise methods.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 140–142, 250; companion slides Ch.4 & Ch.6. Exercise 6.20.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- مفارقة الإنتاجية العكسية (Inverse Productivity Paradox): مقياس أسطر الكود يظهر المبرمج الذي يكتب كوداً متضخماً في لغة دنيا وكأنه أكثر إنتاجية من مبرمج ينجز نفس الوظيفة بأكواد مختصرة في لغة عالية المستوى.

Q6.D08. State Halstead's foundational token classification, and define the core Software Science metrics: Length (N), Vocabulary (η), Volume (V), Potential Volume (V^*), Program Level (L), Difficulty (D), Effort (E), and Estimated Time (T).

Halstead's token counts: η_1, η_2 are the distinct operators and operands; N_1, N_2 are their total occurrences.

Core Software Science Metrics:

1. **Vocabulary and length:** $\eta = \eta_1 + \eta_2$; $N = N_1 + N_2$.
2. **Estimated length:** $\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$.
3. **Volume and potential volume:** $V = N \log_2 \eta$; $V^* = (2 + \eta_2^*) \log_2 (2 + \eta_2^*)$.
4. **Level and difficulty:** $L = V^*/V \approx (2/\eta_1)(\eta_2/N_2)$; $D = 1/L \approx (\eta_1/2)(N_2/\eta_2)$.
5. **Effort and time:** $E = V/L = D \times V$; with Stroud's $S = 18$ discriminations/second, $T = E/S$ seconds.
6. **Language level:** $\lambda = L \times V^* = L^2 V$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 255–262; Companion Slides 11–23. Exercises 6.4, 6.5, 6.6.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- احفظ ثنائية هالستيد: $L = 1/D$. الصعوبة تعتمد طردياً على نصف المعاملات الفريدة $(\eta_1/2)$ ومعدل استخدام المعاملات (N_2/η_2).
- ثابت ستروود $S = 18$ يقسم عليه الجهد العقلي E لاستخراج الزمن بالسكوند.

Q6.D09. Detail the token counting conventions in Halstead's Software Science for block-structured procedural languages (such as C).

Token Counting Conventions for the C Language:

1. **Operators:**
2. **Arithmetic, Relational, and Logical Operators:** `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `&&`, `||`, `!`.
3. **Assignment Operators:** `=`, `+=`, `-=`, etc.
4. **Control Keywords:** `if`, else, for, while, do, switch, case, break, return, goto.
5. **Delimiters and Punctuation (counted in matching pairs as one operator):** Parentheses `( )`, brackets `[ ]`, braces `{ }`, semicolon `;`, comma `,`.
6. **Function Calls:** A function invocation is treated as an operator, with the function name acting as the operator token.
7. **Operands:**
8. **Identifiers:** Variable names, array names, struct field names, pointer names.
9. **Constants and Literals:** Numeric constants (e.g., `0`, `1`, `3.14`), character literals (e.g., `'a'`), string literals (e.g., "Hello").
10. **Type Names in Declarations:** Primitive type keywords (int, float, char) in declaration statements are treated either as operators (specifying storage allocation) or excluded depending on whether only executable statements are tokenized. Under Aggarwal's convention, declarations are tokenized as operators for type specifiers and operands for declared identifiers.
11. **Excluded Tokens:**
12. Comments (`/* ... */`, `// ...`) are completely excluded.
13. Preprocessor directives (#include, #define) are excluded from procedural token counts.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 262–265; Companion Slides 24–38. Exercise 6.10.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- في لغة C، كل زوج أقواس `()` أو `{}` يُعد معاملًا واحداً (Single Operator).
- اسم الدالة المستدعاة يُعامل كمعامل (Operator)، بينما المتغيرات الممررة إليها تعامل كمعاملات عليها (Operands). التعليقات تستبعد كلياً.

Q6.D10. Discuss the empirical criticisms and validity limitations of Halstead's Software Science in modern software engineering.

Critical Analysis of Halstead's Software Science:

1. **Psychological Assumptions and the Stroud Constant:** Halstead assumed that programming effort is governed by a fixed neurological constant ($S = 18$ mental decisions/second). Cognitive psychology research demonstrates that human problem solving is not a steady sequence of uniform binary discriminations, but involves complex mental chunking, domain expertise, and non-linear memory retrieval.
2. **Ignoring Structural Control Flow and Architecture:** Software Science treats code strictly as a flat sequence of lexical tokens. A program with fifty sequential assignment statements yields similar volume and effort to code containing deeply nested conditional loops and recursion, despite the latter being vastly harder to understand and test.
3. **Relationship to LOC:** Halstead measures are derived from operator and operand counts, so their values are related to the amount of source code. No universal correlation coefficient or claim of no added predictive value is established by the inspected course material.
4. **Inadequacy for Component-Based and Object-Oriented Development:** In modern CBD and OO architectures, complexity resides in interface contracts, inheritance hierarchies, dynamic dispatch, and third-party library orchestration—none of which are captured by token counting.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 261–262; companion slides 23. Exercise 6.4.

Section 3: Data Structure, Control Flow & Information Flow Metrics

Q6.D11. Define the data structure metrics family: Variable Count ($VARS$), Live Variables (LV), Variable Span (SP), Module Weakness (WM), and Program Weakness (WP).

Data structure metrics quantify program complexity by analyzing how data elements are declared, referenced, and retained in memory across program execution statements.

Core Metric Definitions:

1. **Amount of Data ($VARS$ / η_2):** The total number of distinct variables declared or referenced within a module.
2. **Live Variables (LV):** Using the **after-statement convention** in the worked trace, a variable is live immediately after executing S_i if it has a value and will be referenced by a later statement before being redefined. $LV(S_i)$ counts those variables at that point. State the convention before tracing because entry-state and exit-state counts differ.
3. **Average Live Variables ($\overline{LV}$):** The sum of live variables across all executable statements in a module divided by the number of executable statements (n):

$$\overline{LV} = \frac{1}{n}\sum_{i=1}^{n} LV(S_i)$$

4. **Average Life of Variables (γ):** The sum of live variables across all statements divided by the total count of distinct variables in the module:

$$\gamma = \frac{\sum_{i=1}^{n} LV(S_i)}{\text{Total Variables}}$$

5. **Variable Span (SP):** The number of statements that intervene between two successive references to the same variable. If variable x is referenced at statement S_a and next referenced at statement S_b, the span is $SP = b - a - 1$.
6. **Module Weakness (WM):** A composite metric measuring module data vulnerability, defined in Aggarwal (scan pp. 278–280, slide 68) as the product of average live variables and average variable life:

$$WM = \overline{LV} \times \gamma$$

7. **Program Weakness (WP):** The system-wide data vulnerability, computed as the arithmetic mean of module weaknesses across all m modules in the program:

$$WP = \frac{1}{m}\sum_{i=1}^{m} WM_i$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 273–280; Companion Slides 39–68. Exercises 6.12, 6.13, 6.15.

Q6.D12. Explain Henry and Kafura's Information Flow Metric (HKM), its parameter definitions, and the slide boundary conventions for top-level and leaf modules.

Henry and Kafura's Information Flow Metric (HKM) measures system structural complexity based on the volume of information flowing into and out of each module.

Governing Equation (Pressman 5e §19.4.2, printed p. 524):

$$HKM(i) = \text{length}(i) \times [fan_in(i) + fan_out(i)]^2$$

Parameter Definitions: - $\text{length}(i)$**:** The internal procedural size of module i, typically measured in Lines of Code (LOC). - $fan_in(i)$**: The number of components that call or pass control and data to module i, plus the number of global data structures from which module i retrieves information.** - $fan_out(i)$**: The number of subordinate components invoked by module i, plus the number of global data structures that module i updates or modifies.** - **The Squared Factor $[fan_in + fan_out]^2$:** Represents the quadratic growth of potential communication channels and interface defects.

Course Slide 74 Boundary Conventions:

In Aggarwal's companion slide 74, specific procedural conventions are established for edge cases: 1. **Top-Level Module Boundary:** For root components whose raw $fan_in = 0$, the procedure assigns $fan_in = 1$ to prevent the product from collapsing or behaving degenerately. 2. **Leaf Module Boundary:** For terminal leaf components that call no other modules ($fan_out = 0$), the procedure assigns $fan_out = 1$. 3. **Reuse Limitation Warning:** Slide 74 explicitly notes that basic fan-in models can inadvertently penalize highly reusable utility components by assigning them artificially massive complexity scores due to widespread incoming calls.

Source: Pressman, R., 5th ed., printed p. 524; K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 69–77. Exercise 6.17.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- صيغة برسمان لهنري وكافورا: $HKM = \text{length} \times [fan_in + fan_out]^2$.
- اصطلاح شريحة أغروال (Slide 74): الوحدات الجذرية العليا التي ليس لها استدعاء وارد يعطى لها $fan_in = 1$ افتراضياً، والوحدات الطرفية التي لا تستدعي غيرها يعطى لها $fan_out = 1$.

Q6.D13. Describe McCabe's Cyclomatic Complexity and its three source-supported computation methods. State the source status of Essential and Design Complexity.

McCabe's Cyclomatic Complexity ($V(G)$) is a graph-based measure of the number of linearly independent paths in a Control Flow Graph. For a graph with P connected components:

1. **Edge–node method:** $V(G) = E - N + 2P$.
2. **Predicate-node method:** for one connected program graph, $V(G) = P_{pred} + 1$.
3. **Region method:** $V(G) = R$, counting all planar regions including the unbounded outer region.

The available course pages explain cyclomatic complexity, but the approved local source set inspected for this pack did not verify usable definitions and calculation rules for **Essential Complexity** and **Design Complexity**. Those two requested terms remain documented source gaps here.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 422–425; companion slides 132–134.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- طرق حساب $V(G)$ الثلاث مدعومة بالمصدر. تعريفات Essential وComplexity Design بعدها فجوة مصدرية محلية، فلا تحفظ تعريفاً غير موثق من هذا الملف.

Q6.D14. What is the verified source status of Knot Count and Nesting Depth in the approved materials for this pack?

Knot Count and Nesting Depth appear in the requested vocabulary, but the inspected course pages and approved local supporting sources did not yield definitions and calculation rules adequate for a reliable answer. Both remain documented gaps; no numeric nesting cutoff or “zero knots” rule is asserted here.

Source status: no verified answer anchor in the approved local source set inspected for this pack.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- هذا توثيق فجوة مصدرية، مو جواب نهائي للمصطلحين. نحتاج مصدر محلي مطابق قبل إدخال تعريف أو قاعدة رقمية.

Q6.D15. Detail the six metrics of the Chidamber & Kemerer (CK) Object-Oriented metric suite, stating the formula and architectural intent of each.

The Chidamber–Kemerer (CK) suite contains six object-oriented measures. Interpret values using the stated variant, counting rules, and a suitable comparison context:

1. **Weighted Methods per Class (WMC):**
2. **Definition:** The sum of the complexities of all methods declared within a class:

$$WMC = \sum_{i=1}^{n} c_i$$

3. If all method complexities c_i are normalized to 1, $WMC = n$ (simple method count).
4. **Intent:** Describes the sum of method complexities under the selected weights; its implications depend on the class and comparison context.
5. **Depth of Inheritance Tree (DIT):**
6. **Definition:** The maximum path length from the target class node to the root of the class hierarchy.
7. **Intent:** Describes inheritance depth; its benefits and costs depend on the hierarchy.
8. **Number of Children (NOC):**
9. **Definition:** The number of immediate subclasses directly subordinate to the target class in the hierarchy.
10. **Intent:** Describes the number of immediate subclasses; it does not alone measure reuse or testing effort.
11. **Coupling Between Objects (CBO):**
12. **Definition:** The count of other classes to which a target class is coupled (a class is coupled to another if it invokes methods or accesses attributes of the other class).
13. **Intent:** Describes dependency breadth; inspect the dependencies before inferring design or maintenance consequences.
14. **Response For a Class (RFC):**
15. **Definition:** The size of the response set of a class, encompassing all methods declared in the class (M) plus all external methods directly invoked by those methods (R):

$$RFC = |M \cup \bigcup_{m \in M} R_m|$$

16. **Intent:** Describes the potential response set used when reasoning about class behavior and test scope.
17. **Lack of Cohesion in Methods (LCOM):**
18. **Definition:** Measures method dissimilarity based on shared instance variable access. Let P be the number of method pairs sharing no instance variables, and Q be the number sharing at least one:

$$LCOM = \begin{cases} P - Q & \text{if } P > Q \\ 0 & \text{otherwise} \end{cases}$$

19. **Intent:** Higher values indicate less measured sharing under this variant; the value alone does not prescribe a class split.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–293; companion slides 80–85. Exercises 6.26, 6.27.

Q6.D16. Outline the MOOD (Metrics for Object-Oriented Design) and Lorenz & Kidd metric suites.

The MOOD (Metrics for Object-Oriented Design) Suite (Brito e Abreu, 1995):

MOOD provides systemic, mechanism-level percentage metrics (bounded between 0% and 100%) to evaluate overall OO system design: 1. **Method Inheritance Factor (MIF) & Attribute Inheritance Factor (AIF):** The ratio of inherited methods/attributes to total available methods/attributes. 2. **Method Hiding Factor (MHF) & Attribute Hiding Factor (AHF):** The percentage of methods/attributes that are private or protected versus public. High hiding indicates effective information hiding. 3. **Coupling Factor (CF):** The percentage of actual non-inheritance inter-class coupling connections relative to the maximum possible connections in the system. 4. **Polymorphism Factor (PF):** The ratio of overridden methods to the total possible polymorphic opportunities.

Lorenz and Kidd Metrics Suite (1994):

Focuses on pragmatic size, inheritance, and internal method organization: 1. **Class Size Metrics:** Number of public methods, number of private operations, total attribute count. 2. **Method Size Metrics:** Average lines of code per method, average cyclomatic complexity per method, message-passing count. 3. **Inheritance Metrics:** Hierarchy nesting levels, number of methods overridden by subclasses, specialization index.

Source: Pressman, R., 5th ed., printed pp. 658–665 / PDF pp. 686–695; companion slide 88.

Q6.D17. Explain the source-supported Use Case Point components and compare their input with Function Point analysis.

The supplied course material supports the unadjusted Use Case Point components. Actors are weighted as simple (1), average (2), or complex (3); use cases are weighted by transaction count as simple (5), average (10), or complex (15). Compute $UAW = \sum(\text{actor count} \times \text{weight})$, $UUCW = \sum(\text{use-case count} \times \text{weight})$, and $UUCP = UAW + UUCW$.

Use-case sizing starts from actor and use-case descriptions, whereas Albrecht Function Points count specified functional components such as inputs, outputs, inquiries, internal files, and external interfaces. The available course source does not provide the technical/environmental adjustment factors or a supported effort-per-UCP conversion, so those quantities cannot be calculated from this source set.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 295–296; companion slides 89–90. Supporting-source search in the approved local set did not verify the adjustment/effort formulas.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- المحسوم بالمصدر المتاح: أوزان الممثلين وحالات الاستخدام وحساب $UUCP$. عوامل التعديل وتحويله إلى ساعات تبقى فجوة موثقة، فلا نعاملها كجزء مثبت من مادة أغروال.

Section 5: Quality, Process & Organizational Metrics

Q6.D18. Define Defect Density, Defect Removal Efficiency (DRE), Phase Containment Effectiveness (PCE), and Mean Time Between Failures (MTBF).

Quality and Defect Metrics Definitions:

1. **Defect Density:**
2. **Definition:** The number of confirmed defects identified in a software product normalized by its physical or functional size:

$$\text{Defect Density} = \frac{\text{Total Defects}}{KLOC} \quad \text{or} \quad \frac{\text{Total Defects}}{\text{Function Points}}$$

3. **Usage:** Benchmarking code quality across releases and project teams.
4. **Defect Removal Efficiency (DRE):**
5. **Definition:** The percentage of defects caught and removed internally during development before product release:

$$DRE = \frac{E}{E + D}$$

   Where E is the number of defects found internally before delivery, and D is the number of defects found by users post-delivery.

6. **Usage:** Comparing the share of known defects found before release under a consistent counting policy. A higher DRE does not prove that testing is perfect or that no defects remain.
7. **Phase Containment Effectiveness (PCE):**
8. **Definition:** A phase-containment rate can be operationally defined as the share of defects attributed to phase i that are detected in that same phase:

$$PCE_i = \frac{D_i^{\text{caught}}}{D_i^{\text{total injected}}} \times 100\%$$

9. **Usage:** Compare phase-level detection using a consistently defined denominator. The formula above is the operational convention used in these practice questions; the cited Mall pages describe phase containment but do not prescribe this exact equation.
10. **Mean Time Between Failures (MTBF):**
11. **Definition:** A statistical reliability metric measuring the mean operational uptime between consecutive system failures:

$$MTBF = MTTF + MTTR$$

    Where $MTTF$ is Mean Time To Failure (uptime) and $MTTR$ is Mean Time To Repair (downtime).

> *Source: Pressman, R., 5th ed., printed pp. 98, 126; K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 308; companion slides Ch.6 & Ch.7; Mall, 4th ed., PDF pp. 84–85 (phase-containment concept only).*

Q6.D19. Explain how metric targets can distort behavior and identify relevant professional-ethics principles.

> **A metric can distort behavior when rewards depend on one narrow target. For example, a LOC bonus can encourage unnecessary code volume, while a closed-ticket target can encourage prioritizing easy tickets over difficult defects. These are scenario-based risks, not outcomes guaranteed by the metric.**
>
> **Professional ethics:** The software-engineering code identifies duties to the public, client and employer, product, professional judgment, management, the profession, colleagues, and oneself. For a metrics program, explain the measure's purpose, use relevant context, and avoid treating one proxy as a complete judgment of an engineer or product.
>
> **Practical check:** Ask whether a target measures the intended outcome and whether its incentives could encourage gaming. Combine evidence and interpret results in context; a single number does not prove quality or effort.
>
> *Source: Sommerville, 9th ed., §1.2, printed pp. 14–16. The LOC and ticket examples are authored illustrations.*

Explain the hierarchical levels of GQM without looking at the text.
Recite Halstead's core formulas ($N, \eta, V, V^*, L, D, E, T, \lambda$) and identify their physical units.
State the three computation methods for McCabe's Cyclomatic Complexity.
Differentiate between Tight Class Cohesion (TCC) and Loose Class Cohesion (LCC).
Explain why tying developer bonuses to LOC causes measurement dysfunction.